

QUESTION 7(a)

Question:

Process Synchronization and Child Process Monitoring Using waitpid().

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("==== Process Synchronization ====\\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        /* Child process */
        printf("\\nChild Process Started\\n");
        printf("Child PID: %d\\n", getpid());

        sleep(3);

        printf("Child Process Completed\\n");

        exit(0);
    }
    else
    {
        /* Parent process */
        printf("\\nParent Process Started\\n");
        printf("Parent PID: %d\\n", getpid());
    }
}
```

```

    /* Parent process */
    printf("\nParent Process Started\n");
    printf("Parent PID: %d\n", getpid());
    printf("Monitoring Child PID: %d\n", pid);

    printf("Parent waiting for child...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("\nChild terminated normally.\n");
        printf("Child Exit Status: %d\n",
               WEXITSTATUS(status));
    }
    else
    {
        printf("\nChild terminated abnormally.\n");
    }

    printf("Parent Process Completed\n");
}

return 0;
}

```

Output

```

sri_dhruthi@Dhruthi:~$ nano skill7a.c
sri_dhruthi@Dhruthi:~$ gcc skill7a.c -o skill7a
sri_dhruthi@Dhruthi:~$ ./skill7a
===== Process Synchronization =====

Parent Process Started
Parent PID: 1190
Monitoring Child PID: 1191
Parent waiting for child...

Child Process Started
Child PID: 1191
Child Process Completed

Child terminated normally.
Child Exit Status: 0
Parent Process Completed

```

QUESTION 7(b)

Question:

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX 200

int main()
{
    char command[MAX];
    char *path;
    char *path_copy;
    char *directory;

    printf("==== Command Path Resolver =====\n");

    printf("Enter command name: ");
    scanf("%199s", command);

    /* Retrieve PATH variable */
    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    /* Make a copy because strtok modifies the string */
    path_copy = malloc(strlen(path) + 1);

    if (path_copy == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }
}
```

```

/* Make a copy because strtok modifies the string */
path_copy = malloc(strlen(path) + 1);

if (path_copy == NULL)
{
    printf("Memory allocation failed.\n");
    return 1;
}

strcpy(path_copy, path);

/* Parse PATH directories */
directory = strtok(path_copy, ":");

while (directory != NULL)
{
    char full_path[MAX];

    snprintf(full_path,
             sizeof(full_path),
             "%s/%s",
             directory,
             command);

    /* Check if executable exists */
    if (access(full_path, X_OK) == 0)
    {
        printf("\nCommand found!\n");
        printf("Executable: %s\n", full_path);

        free(path_copy);
        return 0;
    }

    directory = strtok(NULL, ":");
}

```

```

sri_dhruthi@Dhruthi:~$ nano skill7b.c
sri_dhruthi@Dhruthi:~$ gcc skill7b.c -o skill7b
sri_dhruthi@Dhruthi:~$ ./skill7b
==== Command Path Resolver ====
Enter command name: ls

Command found!
Executable: /usr/bin/ls
sri_dhruthi@Dhruthi:~$ |

```